

Wasserstein distributional robust optimization

Phuong L. Vo

International University

September 26, 2026

Outline

1. Motivation: Why Standard Machine Learning (ERM) Fails

Standard ML (ERM)

Core Assumption:

$$\mathbb{P}_{\text{train}} = \mathbb{P}_{\text{test}}$$

Objective:

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(\theta, x_i)$$

Flaw: Brittle against distribution shifts, fading channels, or noise.

Real-World Shift

Out-of-Distribution:

Dynamic noise, channel fading, dynamic user mobility.

Overconfidence: Deep nets output high confidence on corrupted or shifted inputs.

Intuitive Analogy

Rote Memorization:

ERM is like a student memorizing answer keys.

The WDRO Edge:

WDRO prepares for all plausible question variations within budget ϵ .

2. Earth Mover's Distance: Geometry-Aware Metric

Wasserstein- p Distance Definition

$$W_p(P, Q) = \left(\inf_{\gamma \in \Pi(P, Q)} \mathbb{E}_{(x, y) \sim \gamma} [\|x - y\|^p] \right)^{1/p}$$

Why Not KL-Divergence?

- **Disjoint Support Failure:** If P and Q have disjoint support, $D_{\text{KL}}(P \parallel Q) = \infty$.
- Ignores spatial geometry of the underlying metric space.

Why Wasserstein Wins

- **Geometry-Aware:** Explicitly penalizes point transport via distance norm $\|x - y\|$.
- **Smooth Gradients:** Provides informative gradients even with non-overlapping support.

3. The Primal WDRO Formulation & Computational Bottleneck

The Primal Min-Max Optimization Problem

$$\min_{\theta} \sup_{P \in \mathcal{B}_{\epsilon}(P_n)} \mathbb{E}_{X \sim P}[\ell(\theta, X)]$$

$$\text{where } \mathcal{B}_{\epsilon}(P_n) = \{P : W_p(P, P_n) \leq \epsilon\} \quad \text{and} \quad P_n = \frac{1}{n} \sum_{i=1}^n \delta_{x_i}$$

Primal Components:

- P_n : Empirical training distribution.
- $\mathcal{B}_{\epsilon}(P_n)$: Wasserstein ball of uncertainty around sample data.

Why Is the Primal Intractable?

- **Infinite Dimensions:** $\mathcal{B}_{\epsilon}(P_n)$ contains an infinite continuous measure space.
- Standard SGD cannot search over measure spaces directly.

4. Step-by-Step Derivation of the WDRO Dual Problem

Step 1: Form Lagrangian with Dual Multiplier $\lambda \geq 0$

$$\sup_P \mathbb{E}_P[\ell(\theta, X)] - \lambda (W_p(P, P_n)^p - \epsilon^p) = \lambda \epsilon^p + \sup_P (\mathbb{E}_P[\ell(\theta, X)] - \lambda W_p(P, P_n)^p)$$

Step 2: Apply Kantorovich Duality & Decouple Samples

$$\sup_P (\mathbb{E}_P[\ell(\theta, X)] - \lambda W_p(P, P_n)^p) = \frac{1}{n} \sum_{i=1}^n \sup_{x \in \mathcal{X}} (\ell(\theta, x) - \lambda \|x - x_i\|^p)$$

Step 3: Exact Finite-Dimensional Dual Reformulation

$$\min_{\theta} \inf_{\lambda \geq 0} \left\{ \lambda \epsilon^p + \frac{1}{n} \sum_{i=1}^n \sup_{x \in \mathcal{X}} (\ell(\theta, x) - \lambda \|x - x_i\|^p) \right\}$$

5. Anatomy of the Dual Problem: Why Does It Work?

1. Radius Penalty ($\lambda\epsilon^p$)

Regularization cost proportional to uncertainty budget ϵ , preventing the adversary from shifting samples infinitely far.

2. Robust Evaluation

$$\sup_x [\ell(\theta, x) - \lambda \|x - x_i\|^p]$$

Finds worst-case sample x^* subject to transport penalty $\lambda \|x - x_i\|^p$.

3. Joint Outer Min

$$\min_{\theta, \lambda \geq 0}$$

Trains model weights θ against worst-case samples x^* using standard backpropagation and SGD.

6. Practical Algorithm: Solving WDRO Dual via Adversarial SGD

3-Step Optimization Loop

Step 1: Inner Search

Run K steps of Projected Gradient Ascent to find worst-case x_i^* :

$$x^{(k+1)} = x^{(k)} + \eta_x \nabla_x \left[\ell(\theta, x^{(k)}) - \lambda \|x^{(k)}\| \right]$$

Step 2: Model Parameter Update

$$\theta \leftarrow \theta - \eta_\theta \frac{1}{B} \sum_{i=1}^B \nabla_\theta \ell(\theta, x_i^*)$$

Step 3: Dual Multiplier Update

Adjust scalar λ to track radius budget ϵ .

Connection to PGD Training

- Standard PGD uses a hard $\mathcal{L}_\infty$ box constraint.
- WDRO uses a soft transport penalty $\lambda \|x - x_i\|^p$, allowing optimal out-of-distribution generalization.

7. PyTorch-Style Implementation Recipe for WDRO Dual

```
import torch
```

```
def wdroloss_step(model, x_batch, y_batch, optimizer, criterion, lambda_dual = 10.0, inner_steps = 5, lr_x = 0.01) :
```

```
1. Initialize adversarial sample at current input  $x_a dv =$ 
```

```
 $x_{batch}.clone().detach().requires_grad_ (True)$ 
```

```
2. Inner Maximization: Find  $x^*$  maximizing  $[\text{Loss}(x) - \lambda \cdot \|x - x_i\|^2]$  for
```

```
in range(inner_steps) :  $pred = model(x_a dv)$   $loss_{sem} =$ 
```

```
 $criterion(pred, y_{batch})$   $penalty = \lambda_{dual} * torch.norm(x_a dv -$ 
```

```
 $x_{batch}, p = 2, dim = 1).pow(2).mean()$   $obj_{inner} = loss_{sem} - penalty$ 
```

```
 $grad_x = torch.autograd.grad(obj_{inner}, x_a dv)[0]$   $x_a dv =$ 
```

```
 $(x_a dv + lr_x * grad_x).detach().requires_grad_ (True)$ 
```

```
3. Outer Minimization: Update Model Weights  $\theta$ 
```

```
 $optimizer.zero_grad()$   $outer_{loss} =$ 
```

```
 $criterion(model(x_a dv), y_{batch})$   $outer_{loss}.backward()$   $optimizer.step()$ 
```

```
return  $outer_{loss}.item()$ 
```

8. WaSeCom: Applying WDRO Dual to Wireless Semantic Systems

Wireless Channel Shift Problem

- Wireless semantic transceivers trained on offline channel samples P_n fail under non-stationary fading channels.
- Real-time fine-tuning creates excessive feedback overhead and latency.

WaSeCom Dual Solution

- Evaluates worst-case channel perturbations h^* during offline training using the WDRO dual penalty.
- Provides **zero-shot robustness** under runtime fading without needing online retraining.

9. Lecture Summary & Core Reference Sheet

1. Primal to Dual

Kantorovich duality transforms infinite-dimensional measure search into sample-wise penalization.

2. Algorithm Recipe

Solve via inner ascent on input perturbations x combined with outer descent on network parameters θ .

3. Real-World Impact

Empowers wireless communication deep learning models to handle dynamic distribution shifts reliably.

References